

DIA 8 · SESP/MT

Do NotebookLM ao MVP no Ar — Prompt Engineering e Deploy Automático

Caderno de aprofundamento: transforme contexto e sprints do NotebookLM em um MVP funcional, versionado no GitHub e hospedado no GitHub Pages — com testes e PWA offline.

Curso de Capacitação SESP/MT · Professor Renato Rosa · Módulo 8

§ 1

Agenda do Dia 8 — Do NotebookLM ao MVP no Ar

Hoje é o dia em que o projeto sai do papel e começa a virar código. Você vai aprender a transformar o contexto e as sprints que o NotebookLM gerou em um MVP funcional, hospedado no GitHub Pages — tudo em TypeScript, com PWA e testes automatizados.

BLOCO	O QUE VAMOS FAZER
Manhã (08h–12h)	Validar os prompts do NotebookLM · Configurar o projeto no Antigravity · Escrever as primeiras linhas de código do MVP
Tarde (13h–17h)	Publicar no GitHub · Configurar deploy automático · Próximos passos e prompts para cada sprint

Checklist: você trouxe tudo?

1 Contexto importado do NotebookLM

Abra o **contexto.md** que você gerou no NotebookLM e tenha-o visível no AntigraVity (abra como arquivo).

2 Sprints definidas no NotebookLM

Abra o **sprint.md** com as sprints técnicas em JS/TS/Python PWA. Ele é o seu roteiro de execução.

3 Conta no GitHub

Você precisa de uma conta em **github.com**. Sem ela não é possível publicar.

4 AntigraVity conectado ao GitHub

Se ainda não conectou, volte ao caderno do Dia 6 e siga a seção 'Conectar GitHub'.

DICA

Se você não tem certeza se o AntigraVity está conectado ao GitHub, clique no ícone do painel de Controle do Código-Fonte (escaneie o QR code visual) e veja se aparece o nome do seu repositório.

O Prompt de Contexto — Versão Técnica Aprimorada

O prompt que você usou no NotebookLM provavelmente gerou um resumo amplo. Agora vamos torná-lo mais preciso para extrair informações que o agente de IA consegue transformar diretamente em código.

PROMPT MELHORADO PARA CONTEXTO

Copie e cole no NotebookLM (nova conversa) para gerar um contexto mais técnico e estruturado:

Análise de Requisitos Técnicos — MVP

TXT

Contexto do Projeto

O problema que o sistema resolve

Personas principais e suas necessidades

Dados que o sistema manipula (entidades e campos)

Regras de negócio críticas

Interfaces principais (telas e fluxos)

Integração com sistemas externos

Requisitos não-funcionais (performance, segurança, acessibilidade)

Formato de saída

Responda em português. Para cada seção, seja específico: nomes de entidades, tipos de dados, validações e critérios de aceite mensuráveis.

Evite descrições vagas. Prefira tabelas e listas a parágrafos.

POR QUE RECARREGAR?

O NotebookLM original pode ter perdido o thread da conversa. Uma nova conversa com um prompt mais técnico gera um contexto mais útil para o agente de IA.

§ 4

O Prompt de Sprints — Versão Técnica com GitHub Pages

O sprint.md original pediu sprints em JS/TS/Python PWA. Vamos aprimorar para que cada sprint inclua também o target de publicação no GitHub Pages e a estrutura de branches.

PROMPT MELHORADO PARA SPRINTS

Cole no NotebookLM (mesma conversa do contexto, ou nova):

TXT

```
Gere sprints de execução técnica para o MVP com as seguintes características:
Stack: TypeScript (Vite) + JavaScript + Python (PWA com service worker)
Hospedagem: GitHub Pages (subcaminho /nome-do-repo/)
Versionamento: Git com branches (main + feature/*)
Formato para cada sprint:
1. Nome e objetivo principal
2. Entregas técnicas específicas (arquivos que serão criados/modificados)
3. Critério de aceite (o que precisa estar funcionando ao final)
4. Branch Git associada (ex: feature/setup-inicial)
5. Target de deploy (ex: https://usuario.github.io/nome-repo/)
Considere: autenticação básica, CRUD de entidades principais,
dados sintéticos para mock, testes unitários com Vitest,
```

§ 5

O que é um MVP e por que PWA?

QUANDO USAR

Quando você precisa validar uma ideia com dados reais e usuários de verdade, sem gastar meses em desenvolvimento.

POR QUE

Um MVP (Minimum Viable Product) é a menor versão de um produto que entrega valor real. PWA (Progressive Web App) permite que rodem como app no celular sem passar por loja de aplicativos.

EXEMPLO REAL

O Portal de Pedidos de Compra da SESP: o MVP mostra os pedidos por status, com filtros e totais, em uma URL que qualquer servidor acessa pelo celular.

< 2 sem

para subir o MVP

PWA

roda offline + home screen

GitHub
Pages

hostagem gratuita

Vitest

testes em ms

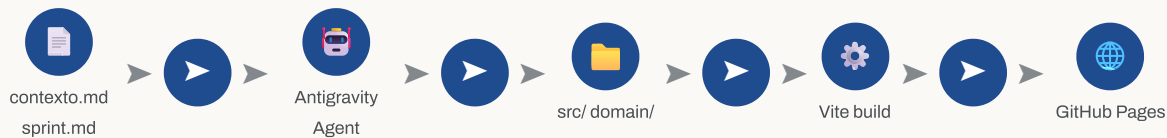
§ 6

A stack do MVP: por que cada peça?

TECNOLOGIA	PAPEL NO MVP	ALTERNATIVA
Vite (vanilla-ts)	Build tool e dev server. Compila TS → JS e empacota para o navegador.	webpack, parcel
TypeScript	Superconjunto de JS com tipos. Reduz erros em tempo de desenvolvimento.	JS puro
Vitest	Framework de testes unitários. Roda no Node, rápido (subsegundo).	Jest, Mocha
vite-plugin-pwa	Gera service worker, manifesto e ícones PWA automaticamente.	workbox manual
Python (scripts)	Gera dados sintéticos (faker) e executa checks de LGPD.	Nodefaker, chance
GitHub Actions	CI/CD: testa, compila e publica no GitHub Pages a cada push.	CircleCI, Travis

§ 7

Arquitetura do projeto — como tudo se conecta



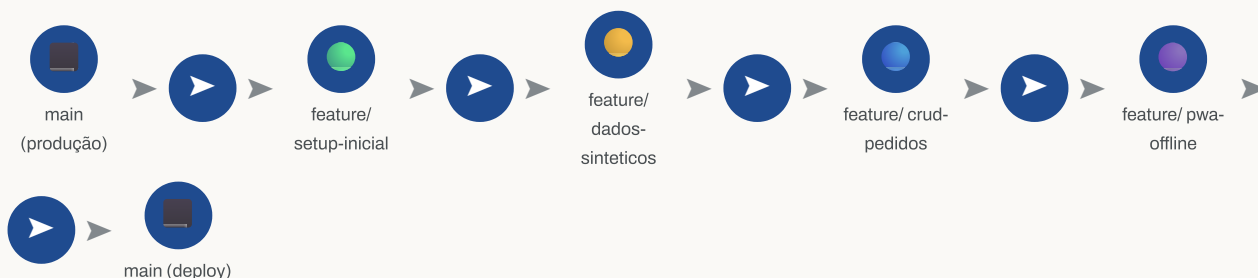
CAMADA	O QUE FICA AQUI	ARQUIVOS 典型ICOS
Dados	JSON estático em public/data/. Gere com script Python.	pedidos.json, usuarios.json
Domínio	Regras de negócio puras, sem dependência de framework.	regras.ts, entidades.ts
Serviços	Chamadas à API (fetch), autenticação.	api.ts, auth.ts
UI	Componentes de apresentação, rotas (hash-based).	main.ts, dashboard.ts
Infraestrutura	Config de build, CI/CD, manifesto PWA.	vite.config.ts, .github/workflows/
Scripts	Geração de dados sintéticos, verificações LGPD.	scripts/gerar_dados.py

§ 8

Modelo de branches — o fluxo Git que você vai seguir

REGRA DE OURO

Cada sprint = uma branch feature. O merge na main só acontece após testes verdes no CI.



main

Branch principal. Código que está em produção (no ar no GitHub Pages). Só recebe merge quando tudo está testado.

feature/*

Branch para cada sprint. Ex: feature/setup-inicial, feature/crud-pedidos. Criada a partir da main.

workflow_dispatch

Gatilho manual no GitHub Actions. Permite publicar mesmo sem push, usando o botão 'Run workflow'.

§ 9

Passo 1 — Criar o repositório no GitHub

1 Acesse github.com/new

Clique em **New repository**. Dê um nome curto e claro (ex: `mvp-pedidos-sesp`). Marque **Public** para ativar GitHub Pages gratuito.

2 Não inicialize

Deixe todas as caixinhas de 'Add README', '.gitignore', etc. deschechadas. Vamos preencher na mão.

3 Copie a URL do repositório

Guarde a URL HTTPS (ex: `https://github.com/seu-usuario/mvp-pedidos-sesp.git`) — você vai usá-la no Antigravity.

NOME DO REPO = BASE PATH

Se o repo se chama `mvp-pedidos-sesp`, a URL do site será `https://seu-usuario.github.io/mvp-pedidos-sesp/`. Anote o nome — você vai precisar dele no `vite.config.ts`.

§ 10

Passo 2 — Scaffold do projeto (sem perder seus arquivos)

ATENÇÃO — PASTA NÃO VAZIA?

Se a sua pasta já tem `contexto.md` e `sprint.md`, o Vite vai recusar criar o projeto nela. Siga o caminho alternativo abaixo.

LAB Caminho normal (pasta vazia)

1 Crie o projeto

No terminal (PowerShell) dentro da pasta do projeto:

```
npm create vite@latest . -- --template vanilla-ts --overwrite
```

2 Instale dependências

```
npm install
```

3 Instale os plugins PWA e testes

```
npm i -D vite-plugin-pwa vitest
```

4 Gere ícones PWA

```
npm i -D @vite-pwa/assets-generator npx pwa-assets-generator --preset minimal-2023 public/icon.svg
```

LAB Caminho alternativo (pasta com contexto.md e sprint.md)

1 Crie em uma subpasta temporária

```
cd .. npm create vite@latest mvp-temp -- --template vanilla-ts mv mvp-temp/* mvp-temp/. * .  
2>/dev/null; rmdir mvp-temp
```

2 Mova seus arquivos de contexto

Os arquivos `contexto.md` e `sprint.md` precisam existir. Se foram movidos, traga-os de volta.

3 Instale as dependências

```
npm install npm i -D vite-plugin-pwa vitest @vite-pwa/assets-generator
```

§ 11

Passo 3 — Configurar o vite.config.ts

O `vite.config.ts` é o arquivo mais importante do projeto. Ele define o caminho base (o nome do seu repo) e ativa o PWA.

```
import { defineConfig } from 'vite'
import { VitePWA } from 'vite-plugin-pwa'
// ➡ TROQUE PELO NOME DO SEU REPOSITÓRIO NO GITHUB
const REPO = 'mvp-pedidos-sesp'
export default defineConfig({
  base: `/${REPO}/`,
  plugins: [
    VitePWA({
      registerType: 'autoUpdate',
      includeAssets: ['favicon.ico', 'apple-touch-icon-180x180.png'],
      manifest: {
        name: 'Portal de Pedidos — SESP/MT (MVP)',
        short_name: 'Pedidos SESP',
        description: 'MVP acadêmico com dados sintéticos — Curso SESP/MT',
        lang: 'pt-BR',
        theme_color: '#1f4b8f',
        background_color: '#ffffff',
        display: 'standalone',
        start_url: '.',
        scope: '.',
        icons: [
          { src: 'pwa-64x64.png', sizes: '64x64', type: 'image/png' },
          { src: 'pwa-192x192.png', sizes: '192x192', type: 'image/png' },
          { src: 'pwa-512x512.png', sizes: '512x512', type: 'image/png' },
          { src: 'maskable-icon-512x512.png', sizes: '512x512', type: 'image/png', purpose: 'maskable' },
        ],
      },
      workbox: { globPatterns: ['**/*.js', '**/*.css', '**/*.html', '**/*.svg', '**/*.png', '**/*.ico', '**/*.json'] },
    }),
  ],
})
```

ERRO MAIS COMUM

Esquecer de trocar `REPO` faz o site abrir em branco no GitHub Pages. O caminho base precisa ser **exatamente** o nome do repositório.

§ 12

Passo 4 — Scripts npm e arquivo de ícone

Adicione os scripts de teste e geração de ícones ao `package.json` :

```
npm pkg set scripts.test='vitest run'
npm pkg set scripts.icones='pwa-assets-generator --preset minimal-2023 public/icon.svg'
```

BASH

Crie o ícone SVG na pasta `public/` (exemplo com as iniciais do seu projeto):

XML

PC

Salve como `public/icon.svg` . Depois execute:

```
npm run icones
```

BASH

§ 13

Passo 5 — A estrutura de pastas src/

REGRA结构性

A pasta `src/` só tem código que roda no navegador. Dados estáticos vão sempre em `public/data/`. Scripts Python ficam em `scripts/`.

```
src/
  domain/
    regras.ts           # regras de negócio puras
    regras.test.ts      # testes das regras
    entidades.ts        # interfaces TypeScript
  auth/
    Papeis.ts           # Roles e permissões
    auth.test.ts
  services/
    api.ts              # chamadas à API (fetch)
    main.ts             # ponto de entrada + router hash
  public/
    data/
      pedidos.json      # dados sintéticos (gerados pelo Python)
      icon.svg
      pwa-*.png         # ícones gerados
  scripts/
    gerar_dados.py      # faker + verificação LGPD
    test_gerar_dados.py # pytest
```

BASH

§ 14

Passo 6 — Gerar dados sintéticos com verificação LGPD

O script Python cria dados realistas para o MVP. Ele mascara CPFs automaticamente e é executado tanto localmente quanto no CI do GitHub Actions.

```

"""Gera dados SINTÉTICOS para o MVP. Nenhum dado real da SESP.
CPF sempre mascarado antes de sair daqui (LGPD)."""
import json, pathlib, random
from datetime import date, timedelta
from typing import NamedTuple
random.seed(42) # reprodutível
SAIDA = pathlib.Path(__file__).resolve().parent.parent / 'public' / 'data'
SAIDA.mkdir(parents=True, exist_ok=True)
UNIDADES = ['Setor de Compras', 'Controladoria', 'Gabinete', 'DIRETORIA de TI']
STATUS = ['rascunho', 'enviado', 'em_cotacao', 'em_auditoria', 'aprovado', 'devolvido']
def mascara_cpf(cpf: str) -> str:
    return f'***.{cpf[3:6]}.***-**'
pedidos = []
for i in range(1, 121):
    pedidos.append({
        'id': i,
        'protocolo': f'PC-2026-{i:04d}',
        'unidade': random.choice(UNIDADES),
        'solicitante_cpf': mascara_cpf(gerar_cpf_valido()),
        'valor_estimado': round(random.uniform(800, 95_000), 2),
        'status': random.choice(STATUS),
        'aberto_em': (date(2026, 1, 2) + timedelta(days=random.randint(0, 240))).isoformat(),
    })
(SAIDA / 'pedidos.json').write_text(json.dumps(pedidos, ensure_ascii=False, indent=2))
print(f'OK: {len(pedidos)} pedidos em {SAIDA}')

```

PYTEST NO CI

O workflow do GitHub Actions executa `python -m pytest -q scripts` para garantir que nenhum CPF foi exposto no JSON gerado.

Passo 7 — Regras de negócio em TypeScript (domínio puro)

```
export interface Pedido {  
  id: number  
  protocolo: string  
  unidade: string  
  solicitante_cpf: string  
  valor_estimado: number  
  status: 'rascunho' | 'enviado' | 'em_cotacao' | 'em_auditoria' | 'aprovado' | 'devolvido'  
  aberto_em: string  
}  
  
/** Valor acima do qual o pedido exige auditoria da Controladoria */  
export const LIMITE_AUDITORIA = 50_000  
  
export function exigeAuditoria(pedido: Pedido): boolean {  
  return pedido.valor_estimado > LIMITE_AUDITORIA  
}  
  
export function totalPorStatus(pedidos: Pedido[]): Record {  
  return pedidos.reduce>((acc, p) => {  
    acc[p.status] = (acc[p.status] ?? 0) + 1  
    return acc  
  }, {})  
}
```

TS

POR QUE DOMÍNIO SEPARADO?

Isolar as regras permite testar sem precisar de um navegador, um servidor ou dados reais. Qualquer pessoa da equipe consegue rodar `npm test` e verificar se as regras estão corretas.

§ 16

Passo 8 — Testes unitários com Vitest

Os testes validam as regras de negócio em milissegundos, sem precisar abrir o navegador. O CI do GitHub Actions exige que todos os testes passem antes de publicar.

```
import { describe, it, expect } from 'vitest'
import { exigeAuditoria, totalPorStatus, type Pedido } from './regras'
const base: Pedido = { id: 1, protocolo: 'PC-2026-0001', unidade: 'Compras',
  solicitante_cpf: '**.123.**-**', valor_estimado: 1000,
  status: 'enviado', aberto_em: '2026-03-01',
}
describe('regras de negócio', () => {
  it('pedido acima de R$ 50 mil exige auditoria', () => {
    expect(exigeAuditoria({ ...base, valor_estimado: 50_000.01 })).toBe(true)
    expect(exigeAuditoria({ ...base, valor_estimado: 50_000 })).toBe(false)
  })
  it('conta pedidos por status', () => {
    const r = totalPorStatus([base, base, { ...base, status: 'aprovado' }])
    expect(r).toEqual({ enviado: 2, aprovado: 1 })
  })
})
```

TS

```
npm test
```

BASH

O QUE FALHA NO CI?

Se qualquer teste falhar, o GitHub Actions impede o deploy. Isso protege você de publicar código quebrado no ar.

§ 17

Passo 9 — O workflow `.github/workflows/deploy.yml`

O workflow é o 'fio terra' entre o seu código e o site no ar. A cada push na branch `main`, ele executa: Python (dados + pytest) → Node (testes + build) → GitHub Pages.

```
name: Deploy no GitHub Pages
on:
  push:
    branches: [main]
  workflow_dispatch:
    # permite acionar manualmente na aba Actions
permissions:
  contents: read
  pages: write
  id-token: write
concurrency:
  group: pages
  cancel-in-progress: true
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - name: Python — gerar dados e verificar LGPD
        uses: actions/setup-python@v7
        with: { python-version: '3.12' }
      - run: pip install -r requirements.txt
      - run: python scripts/gerar_dados.py
      - run: python -m pytest -q scripts
      - name: Node — instalar, testar e compilar
        uses: actions/setup-node@v7
        with: { node-version: '22' }
      - run: npm ci
      - run: npm test
      - run: npm run build
      - uses: actions/configure-pages@v6
      - uses: actions/upload-pages-artifact@v5
        with: { path: dist }
  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
      - id: deployment
        uses: actions/deploy-pages@v5
```

YAML

§ 18

Passo 10 — Ativar GitHub Pages no repositório

1 Configurar no GitHub

No seu repositório, vá em **Settings** → **Pages** → **Source** e selecione **GitHub Actions**. Não é mais necessário escolher branch — o workflow controla isso.

2 Adicionar requirements.txt

Crie um arquivo `requirements.txt` na raiz do projeto com apenas: `pytest` (sem versão, para pegar a última).

WORKFLOW_DISPATCH

Com esse gatilho, você pode acionar o deploy manualmente pela aba **Actions** do repositório, sem precisar dar push. Útil quando você quer verificar o build antes de abrir uma PR.

§ 19

Passo 11 — Seu primeiro deploy

1 Commit e push iniciais

No Antigravity, painel Git: Stage All → Commit (mensagem: 'feat: setup inicial do MVP') → Push.

2 Acompanhe o Actions

Acesse a aba **Actions** do repositório. Você verá o workflow rodando: amarela (running) → verde (sucesso).

3 Acesse o site

Quando o job 'deploy' terminar, o link aparecerá em **Settings** → **Pages**: `https://seu-usuario.github.io/mvp-pedidos-sesp/`.

TEMPO DE PROPAGAÇÃO

Pode levar até 2 minutos para o site aparecer após o deploy terminar. Se der erro 404, aguarde e recarregue. Se persistir por mais de 5 minutos, verifique o log do Actions.

Sprint 0 (Setup) — Prompts para o agente

COMO USAR

Cole cada prompt no painel Agent do Antigravity. Substitua os termos entre colchetes pelo que faz sentido para o seu projeto específico.

PROMPT A — CONFIGURAR O PROJETO DO ZERO

Crie uma estrutura de projeto Vite com vanilla TypeScript. Instale e configure o plugin PWA (vite-plugin-pwa) com manifest para PWA instalável. Configure o base path como `"/nome-do-repo/"` (substitua pelo nome real). Adicione Vitest como dependência de desenvolvimento.

PROMPT B — GERAR O ÍCONE E ÍCONES PWA

Usando a ferramenta pwa-assets-generator com preset minimal-2023, gere todos os ícones PWA (64x64, 192x192, 512x512 e maskable) a partir do arquivo `public/icon.svg`.

PROMPT C — PRIMEIRA ESTRUTURA DE ARQUIVOS

Crie a seguinte estrutura de pastas dentro de `src/`: `domain/` (com `entidades.ts` e `regras.ts`), `auth/` (com `Papeis.ts` e uma função `pode(acao, papel)`), `services/` (com `api.ts` usando `fetch` e `BASE_URL` do `import.meta.env`). Para o campo `status` use `union type literal`.

§ 21

Sprint 1 (Dados) — Prompts para o agente

PROMPT A — SCRIPT PYTHON DE DADOS SINTÉTICOS

Escreva um script Python completo em `scripts/gerar_dados.py` que: (1) use `pathlib` e `json` da `stdlib`; (2) use `random` com `seed 42` para reprodutibilidade; (3) gere 120 pedidos com campos: `id`, `protocolo` (formato `PC-2026-XXXX`), `unidade` (escolha entre 5 opções), `solicitante_cpf` (gere CPF válido e aplique máscara `***.XXX.**-*`), `valor_estimado` (entre 800 e 95000), `status` (escolha aleatório entre 6 opções), `aberto_em` (data entre 2026-01-02 e 2026-09-01); (4) grave em `public/data/pedidos.json` com `indent=2` e `ensure_ascii=False`.

PROMPT B — TESTE DE SEGURANÇA LGPD

Escreva um teste `pytest` em `scripts/test_gerar_dados.py` que: (1) execute o script `gerar_dados.py` via `subprocess`; (2) leia o arquivo `public/data/pedidos.json`; (3) verifique com `regex` que nenhum CPF completo (formato `XXX.XXX.XXX-XX`) está presente no arquivo; (4) afirme que o JSON contém exatamente 120 registros.

PROMPT C — INTERFACE TYPESCRIPT DA ENTIDADE

Com base nos dados do JSON, escreva a interface TypeScript `Pedido` em `src/domain/entidades.ts` com todos os campos e seus tipos exatos. Em `src/domain/regras.ts` escreva: `LIMITE_AUDITORIA = 50000`, função `exigeAuditoria(pedido: Pedido): boolean`, e função `totalPorStatus(pedidos: Pedido[]): Record<string, number>`.

§ 22

Sprint 2 (CRUD) — Prompts para o agente

PROMPT A — SERVIÇO DE API

Em `src/services/api.ts` escreva uma função assíncrona `listarPedidos(): Promise<Pedido[]>` que use `fetch` para carregar `/data/pedidos.json` (use `BASE_URL` do `import.meta.env`). Trate erros com `throw new Error` e texto em português.

PROMPT B — RENDERIZAÇÃO DE LISTA

Em `src/main.ts` escreva uma função `renderLista(pedidos: Pedido[])` que: (1) receba o array de pedidos; (2) monte uma tabela HTML com colunas `Protocolo`, `Unidade`, `Status`, `Valor`, `Aberto em`; (3) use `Intl.NumberFormat` para formatar valores em BRL; (4) use mapeamento de cor por status (`rascunho=cinza`, `enviado=azul`, `em_cotacao=ambar`, `em_auditoria=roxo`, `aprovado=verde`, `devolvido=vermelho`); (5) insira no `#app`.

PROMPT C — ROUTER HASH E INTEGRAÇÃO

Em `src/main.ts`: (1) use `window.addEventListener('hashchange', render)` para detectar navegação; (2) `default` para `#/lista`; (3) importe e chame `listarPedidos`, depois `renderLista`; (4) se `fetch` falhar, mostre mensagem de erro no `#app`; (5) exporte a função para poder ser testada.

§ 23

Sprint 3 (Autenticação e Permissões) — Prompts para o agente

PROMPT A — SISTEMA DE PAPEIS

Em `src/auth/Papeis.ts`: (1) defina tipo `Papel` como `'ADMIN' | 'EDITOR' | 'LEITOR' | 'APROVADOR'`; (2) tipo `Acao` como `'ver' | 'criar' | 'editar' | 'excluir' | 'aprovar' | 'ver_log'`; (3) tabela `PERMISSOES` mapeando cada `Papel` ao array de `Ações` que pode executar; (4) função `pode(papel: Papel, acao: Acao): boolean` que retorna `true` se a ação está no array.

PROMPT B — TESTES DE PERMISSÃO

Em `src/auth/auth.test.ts` escreva 4 testes com `Vitest`: (1) `ADMIN` pode tudo exceto `ver_log`? (não); (2) `LEITOR` não pode editar; (3) `APROVADOR` pode aprovar mas não pode editar; (4) `EDITOR` não pode excluir nem `ver_log`.

PROMPT C — MIDDLEWARE DE ROTA

Escreva uma função `verificarPermissao(papel: Papel, acao: Acao): void` que lance `Error` com mensagem `'Acesso negado: [acao] não permitida para [papel]'` se `pode()` retornar `false`. Exporte como `verificar` from `'../auth/Papeis.ts'`.

§ 24

Sprint 4 (PWA Offline) — Prompts para o agente

PREREQUISITE

O `vite-plugin-pwa` já foi configurado no Passo 3. Estes prompts ajustam o `service worker` para funcionar offline com os dados do projeto.

PROMPT A — CACHE DE DADOS ESTÁTICOS

No `vite.config.ts`, na seção `workbox` do `VitePWA`, configure: `globPatterns` para incluir `public/data/*.json`. Adicione `runtimeCaching` com `Strategy: 'CacheFirst'` para URLs que terminam em `.json`, com `cacheName: 'dados-json'`, e `expiration` `maxEntries: 50`, `maxAgeSeconds: 86400` (1 dia).

PROMPT B — INDICADOR DE MODO OFFLINE

Em `src/main.ts`: (1) escute `window.addEventListener('offline', ...)` para adicionar classe `'offline'` ao `body`; (2) escute `'online'` para remover; (3) no CSS: `body.offline #app::before { content: '⚠️ Modo offline — dados em cache'; display: block; background: var(--ambar); color: white; padding: 8px; text-align: center; font-size: 13px; }`

PROMPT C — REGISTRO MANUAL DO SW (SE NECESSÁRIO)

Se o `VitePWA` não registrar automaticamente (alguns navegadores), escreva em `src/sw-register.ts`: `if ('serviceWorker' in navigator) { window.addEventListener('load', () => navigator.serviceWorker.register('/nome-do-repo/sw.js')); }` e importe no `main.ts`.

Sprint 5 (Visualização e Dashboard) — Prompts para o agente

PROMPT A — KPI CARDS

Em `src/dashboard/kpis.ts`: (1) função `calcularKPIs(pedidos: Pedido[])` que retorna objeto com total de pedidos, total de valores (formatado BRL), quantidade que exige auditoria, percentual de aprovados; (2) função `renderizarKPIs(kpis)` que monte 4 cards HTML (classe `.kpi-card`) e insira no elemento passado como parâmetro.

PROMPT B — GRÁFICO DE BARRAS (VANILLA JS + SVG)

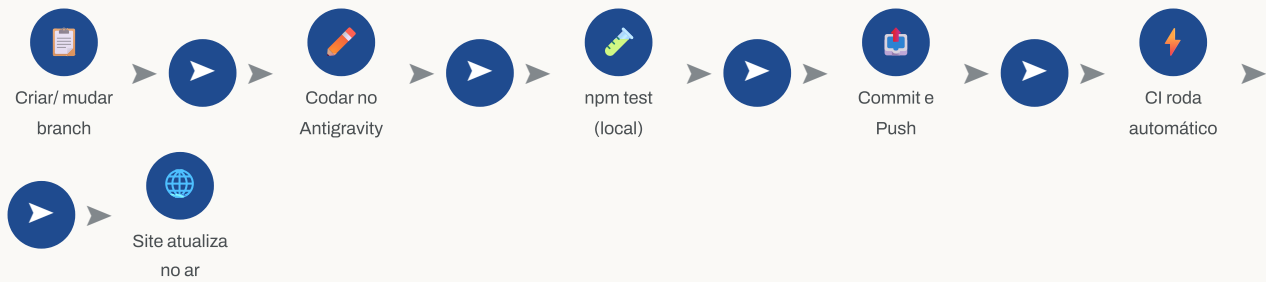
Em `src/dashboard/graficos.ts`: (1) função `renderizarBarras(dados: { label: string; valor: number }[], containerId: string)` que: gere SVG inline com rects proporcionais ao valor máximo; (2) labels em texto abaixo das barras; (3) valores acima das barras; (4) use CSS vars para cores.

PROMPT C — FILTROS POR STATUS

Em `src/dashboard/filtros.ts`: (1) função `renderizarFiltros(onChange: (status: string[]) ⇒ void)` que gere botões/toogle buttons para cada status único; (2) escute click para marcar/desmarcar; (3) chame `onChange` passando array de status ativos; (4) estilize com CSS `.filtro-btn.active { background: var(--accent); color: white }`.

§ 26

O fluxo de trabalho que você vai repetir todo dia



1 Criar branch da sprint

No painel Git do Antigravity: clique no nome da branch atual → 'Create branch' → nomeie como `feature/sprint-2-crud`.

2 Desenvolver e testar localmente

Execute `npm run dev` para ver o site em tempo real no navegador. Execute `npm test` para rodar os testes.

3 Commitar com mensagem descritiva

No painel Agent: 'Análise o que mudou e escreva uma mensagem de commit no conventional commits (feat:, fix:, docs:).' Revise e confirme.

4 Push e aguarde o CI

No painel Git: Sync → empurra a branch. O GitHub Actions começa automaticamente.

5 Mergear na main (após aprovação)

No GitHub: abra uma Pull Request, revise o diff, e clique em 'Merge'. O deploy acontece automaticamente após o merge.

§ 27

Configurar GitHub Pages (via Settings)

1 Acesse Settings do repositório

No github.com, entre no repositório do seu MVP.

2 Pages na barra lateral

Clique em 'Pages' na barra lateral esquerda.

3 Source

Em 'Build and deployment', em 'Source', selecione **GitHub Actions**.

ISSO É SÓ UMA VEZ

Feito isso, o workflow controla 100% dos deploys. Você nunca mais precisa mexer nisso manualmente.

§ 28

Depurando erros de deploy

SINTOMA	CAUSA MAIS PROVÁVEL	SOLUÇÃO
Site abre em branco	base path errado no vite.config.ts	Verifique se REPO = 'nome-exato-do-repo' (sem / no final)
404 em /data/pedidos.json	Falta gerar dados ou JSON vazio	Execute python scripts/gerar_dados.py e commite o public/data/
Testes falham no CI mas passam localmente	Inconsistência entreWindows e Linux (quebras de linha)	Adicione .gitattributes com '* text=auto eol=lf'
workflow_dispatch não aparece	Workflow não foi commitado na main ainda	Faça push da branch main primeiro
deploy-pages falhou com permissões	Pages não foi ativado em Settings	Settings → Pages → Source: GitHub Actions

Checklist de entrega — o que você precisa ter ao final do dia

1 Repositório criado no GitHub

Verifique em `github.com/seu-usuario` se o repo aparece.

2 Projeto scaffoldado no Antigravity

Execute `npm run dev` e confirme que o site abre em `localhost:5173`.

3 vite.config.ts com base path correto

Abra o arquivo e confirme: `base: '/nome-do-repo/'`.

4 Dados sintéticos gerados

Confirme que `public/data/pedidos.json` existe e tem conteúdo.

5 Testes unitários passando

Execute `npm test` — resultado: 'X passed'.

6 Primeiro commit na branch main

No painel Git: stage, commit 'feat: setup inicial', push.

7 Workflow rodou no Actions

Na aba Actions do repo: o workflow deve estar verde.

8 Site acessível no ar

Acesse `https://seu-usuario.github.io/nome-do-repo/` e confirme que abre.

8/8

itens completos?

< 2 min

tempo de deploy

Vitest

testes rodando

PWA

instalável offline

Glossário rápido

PWA (Progressive Web App)

Aplicação web que funciona offline e pode ser instalada na tela inicial do celular. Usa service worker para cache.

Service Worker

Script que roda em segundo plano no navegador, fazendo cache de páginas e dados para funcionar offline.

Vite

Build tool moderno (sucessor do webpack). Compila TypeScript e empacota assets rapidamente.

Vitest

Framework de testes unitários em TypeScript, compatível com Vite, extremamente rápido.

CI/CD (Continuous Integration / Deploy)

Pipeline automático que testa e publica código a cada mudança.

GitHub Actions

Plataforma de CI/CD integrada ao GitHub. Roda scripts na nuvem a cada push.

workflow_dispatch

Gatilho manual do GitHub Actions. Permite rodar o workflow pelo botão 'Run workflow'.

BASE_URL

Variável do Vite que contém o caminho base do projeto (ex: '/mvp-pedidos-sesp/'). Usada para montar URLs de assets.

vite-plugin-pwa

Plugin Vite que gera service worker, manifesto PWA e ícones automaticamente.

pytest

Framework de testes Python. Usado no CI para verificar que os dados sintéticos não expõem CPFs.

§ 31

Quiz rápido — teste seu conhecimento

Qual arquivo define o caminho base do projeto no GitHub Pages?

1. package.json
2. vite.config.ts
3. deploy.yml
4. manifest.webmanifest

► VER RESPOSTA

Por que usamos dados sintéticos no MVP?

1. Para o projeto parecer mais bonito
2. Para não expor dados reais de cidadãos (LGPD)
3. Porque não temos acesso à base de dados real
4. Para o GitHub Actions funcionar

► VER RESPOSTA

O que acontece quando você faz push na branch main?

1. Nada automático
2. O GitHub Actions executa o workflow: testa, compila e publica
3. O site cai
4. O notebooklm atualiza

► VER RESPOSTA

O que é o workflow_dispatch?

1. Um bug do GitHub
2. Um gatilho que permite rodar o workflow manualmente pelo botão
3. Um tipo de branch
4. Um script Python

► VER RESPOSTA

§ 32

Até a próxima — e não pare de codar

Você terminou o Dia 8 com um MVP no ar. Isso significa que:

Código versionado

Tudo que você escreve está no Git, com histórico, branches e capacidade de reverter erros.

Testes automatizados

Qualquer mudança que quebre algo vai aparecer no CI antes de chegar em produção.

Site acessível

Qualquer pessoa com o link consegue acessar — sem instalar nada, direto do celular.

PWA offline

Mesmo sem internet, o site continua funcionando com os dados em cache.

PRÓXIMO PASSO

Na próxima aula, vamos expandir o MVP com mais funcionalidades e aprofundar a engenharia de prompts para que o agente gere código de maior complexidade com menos retrabalho.

Até lá: commit todo dia, rode os testes, e use o painel Agent para perguntar 'como fazer isso?' antes de pesquisar no Google. O agente é o seu primeiro par de olhos técnicos.